

# Plan — Linux GPU box setup for ICONOCRACY 3B pilot

---

## Goal

---

Linux GPU machine to run the prepared ICONOCRACY 3B pilot fine-tuning workflow, then evaluate base vs fine-tuned behavior before deciding on the 7B run.

## Current context

---

- The project already contains the full pilot scaffolding:
  - dataset builder and generated dataset
  - train/validation split files
  - 3B QLoRA pilot config
  - 7B config
  - eval prompt set and eval runner
  - end-to-end pilot shell script
- Relevant prepared files in the repo:
  - `data/training/iconocracy_sft_v1_1.jsonl`
  - `data/training/iconocracy_sft_v1_1_train.jsonl`
  - `data/training/iconocracy_sft_v1_1_val.jsonl`
  - `data/training/iconocracy_eval_prompts_v1_1.jsonl`
  - `tools/configs/training/iconocracy_qwen25_3b_qlora_pilot.json`
  - `tools/configs/training/iconocracy_qwen25_7b_qlora.json`
  - `tools/scripts/train_iconocracy_sft.py`
  - `tools/scripts/run_iconocracy_eval.py`
  - `tools/scripts/run_iconocracy_3b_pilot.sh`
- This plan assumes the remote box is CUDA-capable and reachable over SSH.

## Proposed approach

---

Treat the Linux GPU box as the primary execution environment and keep the local machine as the control surface.

Sequence:

1. Verify remote GPU/CUDA readiness.
2. Clone or sync the repo on the remote machine.
3. Create an isolated Python environment and install training dependencies.
4. Run the 3B pilot end-to-end script.
5. Inspect eval outputs.
6. Decide whether the gain justifies the 7B run.

This avoids cloud quota/provider friction and uses the already-prepared training/eval assets with minimal additional work.

## Step-by-step plan

---

### Phase 1 — Access and remote readiness

1. Collect remote access details:
  - host/IP
  - username
  - preferred workflow: terminal-only or JetBrains remote
2. Verify SSH connectivity from local machine.
3. On the remote host, verify:
  - OS/distribution
  - Python availability
  - CUDA GPU visibility via `nvidia-smi`
  - sufficient disk space

## Phase 2 — Repository setup on the GPU box

1. Clone the repository or update an existing checkout.
2. Confirm the expected files exist, especially:
  - `tools/scripts/run_iconocracy_3b_pilot.sh`
  - `tools/configs/training/iconocracy_qwen25_3b_qlora_pilot.json`
  - `data/training/iconocracy_eval_prompts_v1_1.jsonl`
3. Create a virtual environment in the repo root.
4. Install training dependencies:
  - `torch`
  - `transformers`
  - `datasets`
  - `peft`
  - `accelerate`
  - `trl`
  - `bitsandbytes`
  - `sentencepiece`

## Phase 3 — Sanity checks before training

1. Confirm CUDA visibility inside Python.
2. Confirm the base model can be pulled/downloaded.
3. Confirm the train/val JSONL files are readable.
4. Optionally run a very small smoke test if remote time/cost is a concern.

## Phase 4 — Run the cheap pilot

1. Execute:
  - `bash tools/scripts/run_iconocracy_3b_pilot.sh`
2. Let the script perform:
  - 3B LoRA training
  - base-model evaluation
  - fine-tuned evaluation
3. Record resulting artifact paths, especially:
  - adapter directory
  - base eval JSONL
  - fine-tuned eval JSONL

## Phase 5 — Review and decision

1. Compare base vs fine-tuned outputs using the eval JSONL files.
2. Check for improvement in:
  - terminology discipline
  - method explanation
  - chapter-planning coherence
  - corpus-to-analysis usefulness
  - caution with pipeline claims
3. Decide whether to:
  - proceed to 7B as-is
  - refine dataset again first
  - add more hand-curated examples before scaling up

## Files likely to change

---

Remote execution should not require code changes if the existing scripts run cleanly.

Most likely generated/updated artifacts:

- `~/Models/iconocracy-qwen25-3b-sft-pilot-lora` (remote home directory)
- `data/training/eval_base_qwen3b.jsonl`
- `data/training/eval_ft_qwen3b.jsonl`

Potential files to adjust later only if needed:

- `tools/configs/training/iconocracy_qwen25_3b_qlora_pilot.json`
- `tools/scripts/run_iconocracy_3b_pilot.sh`
- `tools/scripts/train_iconocracy_sft.py`

## Tests / validation

---

### Remote environment validation

- `nvidia-smi` should show the GPU and driver
- Python should import `torch` successfully
- `torch.cuda.is_available()` should be `True`

### Training validation

- Training script starts without config/path errors
- Adapter directory is created
- No immediate OOM or package incompatibility errors

### Evaluation validation

- `eval_base_qwen3b.jsonl` is created
- `eval_ft_qwen3b.jsonl` is created
- Eval outputs cover the prompt IDs from `iconocracy_eval_prompts_v1_1.jsonl`

### Decision validation

The pilot is considered successful enough to justify 7B if the fine-tuned model clearly improves over base on:

- `ENDURECIMENTO` handling

- `Feminilidade de Estado` attribution discipline
- QUAN→QUAL→síntese explanation
- corpus note generation quality
- adherence to ICONOCRACY framing

## Risks / tradeoffs

---

### Risks

- Remote box may have incompatible CUDA/driver/PyTorch versions
- `bitsandbytes` may need adjustment depending on the GPU/driver stack
- The 3B pilot may still look somewhat templated because the dataset is partly programmatically generated
- Storage may be tight if model caches are on a small root disk

### Tradeoffs

- 3B pilot is cheaper and faster, but not the final target quality
- A successful 3B pilot gives directional confidence, not final-model certainty
- Using the existing GPU box reduces infrastructure pain but may require a bit of environment debugging upfront

## Open questions

---

1. What are the remote host details?
2. Is the remote box already provisioned with CUDA/NVIDIA drivers?
3. Is the repo already present there, or will it be cloned fresh?
4. Does the user want terminal-only setup or JetBrains remote integration from the start?
5. Is the goal only the 3B pilot now, or should the remote box be prepared with the 7B run in mind as well?

## Immediate next interaction when resuming

---

Ask for or use:

- remote host/IP
- username
- confirmation of whether JetBrains remote should be part of the initial setup

Then proceed with:

1. SSH connectivity check
2. GPU/CUDA verification
3. repo/env setup
4. pilot script execution